Unit 1 — Self Study Topics

Unit: 1 | **Folder:** self-study | **CO:** CO1, CO2, CO4, CO5

This file covers the self-study topics designated for Unit 1 of the ADSA course. Each section is independent and can be studied after completing the corresponding unit lectures.

Topic 1 — Amortized Analysis

What is Amortized Analysis?

Amortized analysis gives the **average cost per operation** over a sequence of operations, even if individual operations vary in cost. Unlike average-case analysis, it provides a **worst-case guarantee** for the total sequence.

Three Methods

1. Aggregate Method

Compute total cost T(n) for n operations, then amortized cost = T(n) / n.

Example: Dynamic Array (doubling)

- Most `append` operations cost O(1).
- When capacity doubles, a copy of all elements costs O(n).
- Total cost of n appends: O(1 + 2 + 4 + ... + n) = O(2n) = O(n)
- Amortized cost per append: O(n)/n = **O(1)**

2. Accounting (Banker's) Method

Assign each operation an **amortized cost** (possibly higher than actual). The surplus is stored as "credit" to pay for expensive future operations.

```
Dynamic Array Example:
- Charge each append: amortized cost = 3 (actual = 1, store 2 credits)
- On doubling (copy n elements): use 2 credits stored per element
- Credits are always non-negative → amortized cost = O(1) per operation
```

3. Potential Method

Define a **potential function** Φ over the data structure state.

```
Amortized cost = actual cost + ΔΦ

For dynamic array: Φ = 2 × (current size) − capacity
- When half full: Φ = 0
- After n inserts before doubling: Φ = n (sufficient to cover copy)
```

Applications

- Dynamic arrays, hash table resizing, splay trees, skew heaps
- `push/pop` on stack: O(1) amortized even with occasional O(n) resizes

---

Topic 2 — Advanced Recurrence Relations

Solving Recurrences: Master Theorem

For $T(n) = aT(n/b) + f(n)$, where $a \geq 1$, $b > 1$:

Let $c = \log_b(a)$

Condition	Result
$f(n) = O(n^{(c-\varepsilon)})$	$T(n) = \Theta(n^c)$
$f(n) = \Theta(n^c \cdot \log^k n)$	$T(n) = \Theta(n^c \cdot \log^{(k+1)} n)$
$f(n) = \Omega(n^{(c+\varepsilon)})$ and regularity	$T(n) = \Theta(f(n))$

Examples:

```
T(n) = 2T(n/2) + n   → a=2, b=2, c=1, f(n)=n=Θ(n^1) → T(n) = Θ(n log n)
T(n) = T(n/2) + 1    → a=1, b=2, c=0, f(n)=1=Θ(1)   → T(n) = Θ(log n)
T(n) = 4T(n/2) + n   → a=4, b=2, c=2, f(n)=n=O(n^1) → T(n) = Θ(n²)
```

Substitution Method

Guess the form, prove by induction.

```
T(n) = 2T(n/2) + n
Guess: T(n) = O(n log n)
Assume T(k) ≤ ck log k for k < n:
  T(n) = 2T(n/2) + n ≤ 2·c(n/2)log(n/2) + n
       = cn log(n/2) + n = cn(log n − 1) + n
       = cn log n − cn + n ≤ cn log n   for c ≥ 1 ✓
```

Recursion Tree Method

Draw each level of the recurrence; sum costs per level.

```
T(n) = 2T(n/2) + n

Level 0:  n                    (cost n)
Level 1:  n/2  n/2             (cost n)
Level 2:  n/4 n/4 n/4 n/4     (cost n)
...
Level log n: leaves            (cost n)

Total: n × (log n + 1) levels = O(n log n)
```

Topic 3 — Threaded Binary Trees

Motivation

In a standard binary tree, ~50% of child pointers are NULL. **Threaded binary trees** replace NULL pointers with threads to predecessor/successor nodes in the inorder traversal, enabling traversal **without a stack**.

Types

- **Right-threaded:** NULL right pointer → points to inorder successor
- **Left-threaded:** NULL left pointer → points to inorder predecessor
- **Fully-threaded:** Both

Structure

```
class ThreadedNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None
        self.left_thread = False   # True if left is a thread (not real
        self.right_thread = False  # True if right is a thread (not rea
```

Inorder Traversal (no stack, no recursion)

```
def threaded_inorder(root):
    """Inorder traversal of right-threaded BST. O(n) time, O(1) space."
    # Find leftmost node
    curr = root
    while curr.left and not curr.left_thread:
        curr = curr.left

    while curr:
        print(curr.val)

        if curr.right_thread:
            curr = curr.right  # Follow thread to inorder successor
        else:
            # Go to leftmost node of right subtree
            curr = curr.right
            if curr:
                while curr.left and not curr.left_thread:
                    curr = curr.left
```

Advantages

- O(1) extra space for traversal
- Useful in memory-constrained environments

Topic 4 — Red-Black Trees

Properties

A Red-Black Tree is a self-balancing BST where each node has a **color** (red or black) satisfying:

1. Every node is red or black
2. Root is black
3. Every leaf (NIL) is black
4. Red node's children are both black (no two consecutive reds)
5. All paths from any node to its descendant NIL leaves have the **same number of black nodes** (black-height)

Height Guarantee

A Red-Black Tree with n nodes has height $\leq 2 \log_2(n+1) \rightarrow O(\log n)$ for all operations.

Rotations

Same as AVL: left-rotate and right-rotate to restore structure after insertion/deletion.

Insertion (simplified)

```
1. Insert like BST, colour new node RED
2. Fix violations:
   Case 1: Uncle is RED → recolour uncle, parent → BLACK; grandparent →
   Case 2: Uncle is BLACK, zig-zag shape → rotate to convert to Case 3
   Case 3: Uncle is BLACK, line shape → rotate grandparent, swap colour
3. Ensure root is BLACK
```

Red-Black vs AVL

Property	Red-Black	AVL
Height	$\leq 2 \log n$	$\leq 1.44 \log n$
Search	O(log n)	O(log n)
Insert/Delete	O(log n), fewer rotations	O(log n), more rotations
Best for	Insert/delete heavy	Search heavy
Used in	Java TreeMap, Linux scheduler	Databases

Topic 5 — B-Trees and B+ Trees (Extended)

B-Tree (Review & Depth)

(Covered in L39 — this section focuses on B+ Trees and comparison.)

B+ Trees

In a **B+ Tree**, only **leaf nodes** hold data records. Internal nodes hold only keys as

separators/guides.

```
Internal:  [   20  |  40  ]
          /        |       \
Leaf:  [10,15] → [20,30,35] → [40,50,60]
                   ↑ linked list of leaves
```

Key differences from B-Tree:

Property	B-Tree	B+ Tree
Data in	All nodes	Leaves only
Leaf links	No	Yes (linked list)
Range query	O(t log n + k)	O(log n + k) linear
Space efficiency	Less	More (keys duplicated)
Used in	File systems	Databases, RDBMS

B+ Tree Range Query

```
def range_query(tree, low, high):
    """Efficient range query using leaf linked list. O(log n + k)"""
    # 1. Find leaf node containing low (O(log n))
    leaf = tree.search_leaf(low)

    # 2. Scan leaf linked list until > high (O(k))
    results = []
    while leaf:
        for key in leaf.keys:
            if low <= key <= high:
                results.append(key)
            elif key > high:
                return results
        leaf = leaf.next_leaf   # Follow linked list
    return results
```

Topic 6 — Splay Trees

Concept

A **Splay Tree** is a self-adjusting BST where the most recently accessed element is moved to the root via **splaying** (a series of rotations). This gives **O(log n) amortized** performance without storing height or balance information.

Splay Operation (zig, zig-zig, zig-zag)

```
Zig (single rotation):   node is child of root → one rotation
Zig-zig (double same):   node and parent are both left/right children →
Zig-zag (double cross):  node is left child, parent is right (or vice v
```

```
def splay(root, key):
    """Bring node with key to root via splay. Returns new root."""
```

```
    if root is None or root.key == key:
        return root

    if key < root.key:
        if root.left is None:
            return root
        # Zig-zig: key < root.left.key
        if key < root.left.key:
            root.left.left = splay(root.left.left, key)
            root = rotate_right(root)
        # Zig-zag: key > root.left.key
        elif key > root.left.key:
            root.left.right = splay(root.left.right, key)
            if root.left.right:
                root.left = rotate_left(root.left)
        return root if root.left is None else rotate_right(root)
    # Symmetric for right subtree
    ...
```

Properties

- **Amortized O(log n)** per operation
- **Locality of reference**: frequently accessed nodes stay near root
- Used in caches, network routers, text editors

Topic 7 — Huffman Trees

Concept

Huffman coding is an optimal prefix-free compression algorithm. Characters are encoded with variable-length codes — frequent characters get shorter codes.

Algorithm

1. Build a **priority queue** (min-heap) of characters by frequency
2. Repeatedly extract two minimum-frequency nodes, combine into a new internal node with their sum as frequency
3. Repeat until one node remains (the root)
4. Assign `0` to left edges, `1` to right edges

```
import heapq

def huffman_tree(freq_map):
    """Build Huffman tree. Returns root node."""
    heap = [(freq, idx, char) for idx, (char, freq) in enumerate(freq_m
    heapq.heapify(heap)

    counter = len(heap)
    while len(heap) > 1:
        f1, _, left = heapq.heappop(heap)
        f2, _, right = heapq.heappop(heap)
```

```
        node = (f1 + f2, counter, ('internal', left, right))
        heapq.heappush(heap, node)
        counter += 1

    return heap[0][2]

def build_codes(node, prefix='', codebook={}):
    if isinstance(node, str):  # Leaf
        codebook[node] = prefix or '0'
    else:
        _, left, right = node
        build_codes(left, prefix + '0', codebook)
        build_codes(right, prefix + '1', codebook)
    return codebook
```

Example

```
Characters: a(45), b(13), c(12), d(16), e(9), f(5)

Huffman codes:
  a: 0       (1 bit, most frequent)
  b: 101     (3 bits)
  c: 100     (3 bits)
  d: 111     (3 bits)
  e: 1101    (4 bits)
  f: 1100    (4 bits)

Optimal: each code is prefix-free (no code is a prefix of another)
Compression ratio: ~2.24 bits/char vs 3 bits/char (fixed width)
```

Properties

- **Optimal** prefix-free code (proven by greedy exchange argument)
- Time: O(n log n) to build tree
- Used in: gzip, JPEG, MP3, ZIP

Topic 8 — Decision Trees

Concept

A **decision tree** models the execution of a comparison-based algorithm. Each internal node is a comparison; each leaf is an outcome.

Lower Bound on Comparison Sorting

For sorting n elements:

- There are n! possible orderings (leaves in the decision tree)
- A binary decision tree of height h has at most 2^h leaves
- Therefore: $2^h \geq n! \rightarrow h \geq \log_2(n!) = \Omega(n \log n)$

This proves no comparison-based sorting algorithm can do better than O(n log n) in the worst case.

```
# Decision tree for sorting [a, b, c]:
#            a < b?
#           /      \
#       b < c?   a < c?
#      /    \    /    \
#   abc   a<c? bac   bca
#         / \
#        acb cba
```

Applications

- Proving lower bounds for algorithms
- Machine learning (ID3, CART — split by information gain/Gini index)
- Optimal binary search trees (Knuth's algorithm: $O(n^2)$)

Topic 9 — Space-Time Trade-offs

Principle

In algorithm design, **space and time are often exchangeable**:

- Use more memory to precompute results → faster queries
- Use less memory by recomputing → slower but space-efficient

Classic Examples

Technique	Space	Time	Example
Memoization	O(n)	O(n)	Fibonacci, LCS
Lookup tables	O(table_size)	O(1)	Trigonometry, CRC
Prefix sums	O(n)	O(1) range query	Subarray sum queries
Hashing	O(n)	O(1)	Dictionary, set membership
Sorting pre-sort	O(1) extra	O(n log n) then O(1)	Binary search
BFS (graph)	O(V)	O(V + E)	Shortest path (unweighted)

Prefix Sum Example

```
def preprocess(arr):
    """O(n) build: prefix[i] = arr[0] + ... + arr[i-1]"""
    prefix = [0] * (len(arr) + 1)
    for i, v in enumerate(arr):
        prefix[i + 1] = prefix[i] + v
    return prefix

def range_sum(prefix, l, r):
    """O(1) query: sum of arr[l..r]"""
    return prefix[r + 1] - prefix[l]
```

```
# Without prefix sums: O(n) per query
# With prefix sums: O(1) per query after O(n) precomputation
```

References

- Cormen et al., *Introduction to Algorithms*, 4th Ed. (MIT Press, 2022) — Ch. 17 (Amortized), Ch. 13 (Red-Black), Ch. 18 (B-Trees)
- Skiena, *The Algorithm Design Manual*, 3rd Ed. (Springer, 2020)
- Laaksonen, *Guide to Competitive Programming*, 3rd Ed. (Springer, 2024)